

**WEEK 03 MODULE**  
**Programming Practice**

TOPIC : 1 MERGE SORT

a) Easy

que. 2 **What is Sorting?**

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

**Types of Sorting :-**

- i. Bubble Sort
- ii. Selection Sort
- iii. Insertion Sort
- iv. Merge Sort
- v. Quick Sort
- vi. Heap Sort



**Merge Sort**

Definition : Merge Sort is a comparison-based, divide-and-conquer sorting algorithm that sorts an array by repeatedly dividing it into smaller subarrays, sorting those subarrays, and then merging them back together in sorted order. Instead of sorting the entire array at once, Merge Sort breaks the problem into smaller, easier-to-solve parts, solves each part recursively, and combines the results to obtain the final sorted array.

**Key Points:-**

- Uses Divide and Conquer.
- Recursively splits the array.
- Merges sorted subarrays.
- Stable sorting algorithm.

**How It Works:-**

[38, 27, 43, 3]

↓ Divide

[38, 27] [43, 3]

↓ Divide

[38] [27] [43] [3]

↓ Merge

[27, 38] [3, 43]

↓ Final Merge

[3, 27, 38, 43]

### Problem Statement

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets. Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

### 1. Question Analysis

The university's Training and Placement Cell has collected the salary packages offered to students by different companies. Since the package values were entered in the order they were received, the records are not arranged in any particular order.

Before publishing the placement statistics, the department wants all package values sorted in ascending order. As the university plans to manage placement records for thousands of students in the future, an efficient sorting algorithm is required. Therefore, the task is to implement the Merge Sort algorithm to arrange the package values in increasing order and display the final sorted list.

### 2. Problem Analysis

The problem requires sorting an unsorted list of salary package values in ascending order using the Merge Sort algorithm. Since the number of package records can be very large, the chosen algorithm should provide efficient and consistent performance even for large datasets.

Merge Sort solves this problem by recursively dividing the array into smaller subarrays until each subarray contains only one element. These smaller sorted subarrays are then merged back together in the correct order by comparing their elements.

### 3. Algorithm Selection Analysis

Selected Algorithm: Merge Sort

- Merge Sort follows the Divide and Conquer technique.
- It divides the array into smaller subarrays, making the sorting process simpler and more efficient.
- It guarantees a time complexity of  $O(N \log N)$  in the best, average, and worst cases.
- It performs efficiently even for large datasets containing thousands of elements.
- It produces the output in ascending order by merging smaller sorted subarrays.
- It is well-suited for this problem as the university plans to handle large amounts of placement data in the future.
- Hence, Merge Sort is the most appropriate algorithm for solving this problem.

### 4. Points to Remember

- Merge Sort follows the Divide and Conquer technique.
- The array is recursively divided into two halves.
- Division continues until every subarray contains only one element.
- A single element is considered already sorted.
- Sorting actually occurs during the merging process.
- While merging, elements from both subarrays are compared one by one.
- The smaller element is placed first to maintain ascending order.
- Temporary arrays are required during merging.

- Time Complexity
  - Best Case :  $O(N \log N)$
  - Average Case :  $O(N \log N)$
  - Worst Case :  $O(N \log N)$
- Space Complexity :  $O(N)$

## 5. Core Idea / Logic

- Divide the given array into two equal halves.
- Continue dividing each subarray recursively until every subarray contains only one element.
- Consider each single element as an already sorted subarray.
- Merge two adjacent sorted subarrays into a larger sorted subarray.
- During merging, compare the elements of both subarrays one by one.
- Place the smaller element first into the temporary array to maintain ascending order.
- Copy any remaining elements from either subarray after all comparisons are completed.
- Repeat the merging process until all subarrays are combined into one completely sorted array.

## 6. Execution Flow

Part A : Execution Flow

### Input Phase

1. The program starts execution from the main() method.
2. A Scanner object is created to receive user input.
3. The user enters the number of package values (**N**), each of which is validated before being stored in the array.
4. An integer array of size **N** is created.
5. The user enters all package values.
6. Each entered value is stored sequentially in the array.
7. After all inputs are stored, the mergeSort() method is called.

### Processing Phase

1. The mergeSort() method receives the array along with the starting and ending indices.
2. It checks whether the current subarray contains more than one element.
3. If true, the middle index is calculated.
4. The array is divided into two halves.
5. The left half is recursively divided.
6. The right half is recursively divided.
7. This division continues until every subarray contains only one element.
8. Once division is complete, the merge() method begins combining adjacent subarrays.
9. Temporary arrays are created to store the left and right subarrays.
10. Elements from both temporary arrays are compared one by one.
11. The smaller element is copied into the original array.
12. If one temporary array becomes empty, all remaining elements of the other temporary array are copied.
13. The merged portion becomes completely sorted.
14. The same merging process continues recursively until the entire array is sorted.

### Output Phase

1. Control returns to the main() method.
2. The sorted array is traversed from the first element to the last.
3. Each package value is printed in ascending order.
4. The program terminates successfully.

## Part B : Detailed Steps

### Step 1

The program starts execution from the main() method.

### Step 2

The user enters the number of placed students (**N**).

### Step 3

The program checks whether **N** is within the valid range (1 to 100000). If the condition is not satisfied, an error message is displayed and the program terminates.

### Step 4

An integer array of size **N** is created to store the salary package values.

### Step 5

The user enters the salary package values, which are stored in the array one by one, validate each score according to the specified constraints, and store the valid scores in the array.

### Step 6

The mergeSort() method is called by passing the array, starting index (0), and ending index (**N** - 1).

### Step 7

The mergeSort() method checks whether the current subarray contains more than one element (left < right).

### Step 8

If the condition is true, the middle index of the current subarray is calculated.

### Step 9

The array is recursively divided into smaller left and right subarrays until each subarray contains only one element.

### Step 10

The merge() method is called to merge the divided subarrays.

### Step 11

Temporary arrays are created and the elements of the left and right subarrays are copied into them.

### Step 12

The elements of both temporary arrays are compared, and the smaller element is copied back into the original array.

### Step 13

If any elements remain in either temporary array, they are copied into the original array.

#### Step 14

The merging process continues until all subarrays are merged into one completely sorted array.

#### Step 15

Control returns to the main() method.

#### Step 16

The sorted salary package values are displayed.

#### Step 17

The Scanner object is closed and the program terminates successfully.

### 7. Program Structure Analysis

#### i. Variable n

```
int n;
```

##### Purpose:

- Stores the total number of placed students whose salary package values are to be sorted.
  - Determines the size of the array.
  - Defines the range of elements that will be processed by Merge Sort.
- Constraint:

$$1 \leq n \leq 100000$$

means:

$$1 \leq n \leq 100000$$

In simple words:

- Minimum value of n can be 1.
- Maximum value of n can be 100000.

#### ii. Array arr[]

```
int[] arr = new int[n];
```

##### Purpose:

- Stores the salary package values of all placed students.
  - Acts as the main array on which Merge Sort is performed.
  - Holds the final sorted package values after the sorting process is completed.
- Constraint:

$$100 \leq \text{Package} \leq 10000$$

In simple words:

- Minimum Package can be 100.
- Maximum Package can be 10000.

### iii. Variables left, mid and right

```
int left, mid, right;
```

**Purpose:**

- left represents the starting index of the current subarray.
- right represents the ending index of the current subarray.
- mid stores the middle index, which divides the array into two halves.
- These indices help recursively divide and merge the array.

### iv. Variables n1 and n2

```
int n1 = mid - left + 1;  
int n2 = right - mid;
```

**Purpose:**

- Store the sizes of the left and right subarrays.
- Help create temporary arrays of the required size.
- Ensure all elements are copied correctly during the merge process.

### v. Temporary Arrays leftArray[] and rightArray[]

```
int[] leftArray = new int[n1];  
int[] rightArray = new int[n2];
```

**Purpose:**

- Temporarily store the elements of the left and right subarrays.
- Allow safe comparison of elements during merging.
- Help combine two sorted subarrays into one sorted array.

### vi. Variables i, j and k

```
int i = 0;  
int j = 0;  
int k = left;
```

**Purpose:**

- i traverses the left temporary array.
- j traverses the right temporary array.
- k indicates the position in the original array where the next sorted element will be placed.

### vii. Input Loop

```
for(int i = 0; i < n; i++){  
    arr[i] = sc.nextInt();  
}
```

**Purpose:**

- Accepts the salary package values from the user.
- Stores each value in the array.
- Ensures all input values are available before sorting begins.

**viii. Recursive mergeSort() Method**

```
mergeSort(arr, left, right);
```

**Purpose:**

- Recursively divides the array into smaller subarrays.
- Continues dividing until each subarray contains only one element.
- Calls the merge() method to combine the sorted subarrays.

**ix. Conditional Statement**

```
if(left < right)
```

**Purpose:**

- Checks whether the current subarray contains more than one element.
- Prevents unnecessary recursive calls.
- Acts as the base condition for Merge Sort recursion.

**x. Middle Index Calculation**

```
int mid = left + (right - left) / 2;
```

**Purpose:**

- Calculates the middle index of the current subarray.
- Divides the array into left and right halves.
- Uses a safe formula to avoid integer overflow.

**xi. merge() Method**

```
merge(arr, left, mid, right);
```

**Purpose:**

- Merges two sorted subarrays into a single sorted subarray.
- Compares elements from both temporary arrays.
- Places elements in ascending order into the original array.

**xii. Comparison Loop**

```
while(i < n1 && j < n2)
```

**Purpose:**

- Compares the elements of the left and right temporary arrays.
- Places the smaller element into the original array.
- Continues until one of the temporary arrays is completely traversed.

### **xiii. Remaining Elements Loop**

```
while(i < n1)
while(j < n2)
```

#### **Purpose:**

- Copies any remaining elements from the left temporary array.
- Copies any remaining elements from the right temporary array.
- Ensures no element is left unprocessed after merging.

### **xiv. Output Loop**

```
for(int i = 0; i < n; i++)
{
    System.out.print(arr[i] + " ");
}
```

#### **Purpose:**

- Traverses the sorted array.
- Displays all salary package values in ascending order.
- Produces the final sorted output.

### **xv. Another imp terms**

#### **1. Import Statement**

##### **Purpose:**

Imports the Scanner class from the java.util package to accept user input during program execution.

#### **2. Class Declaration**

##### **Purpose:**

Defines the main class that contains all methods required to perform Merge Sort and execute the program.

#### **3. merge() Method**

##### **Purpose:**

Merges two already sorted subarrays into a single sorted array while maintaining ascending order.

##### **Parameters:**

- Array
- Left Index
- Middle Index
- Right Index

##### **Working:**

- Calculates the sizes of both subarrays.
- Creates temporary arrays.
- Copies elements into the temporary arrays.
- Compares elements from both arrays.
- Places the smaller element into the original array.
- Produces a sorted merged subarray.



#### 4. mergeSort() Method

**Purpose:**

Recursively divides the array into smaller subarrays and sorts them using the Divide and Conquer approach.

**Parameters:**

- Array
- Left Index
- Right Index

**Working:**

- Checks the base condition (left < right).
- Calculates the middle index.
- Recursively sorts the left half.
- Recursively sorts the right half.
- Calls the merge() method to combine both sorted halves.

#### 5. main() Method

**Purpose:**

Acts as the entry point of the program and coordinates the overall execution.

**Responsibilities:**

- Creates the Scanner object.
- Reads the value of **N**.
- Creates the array.
- Accepts all package values.
- Calls the mergeSort() method.
- Prints the sorted array.
- Closes the Scanner.

#### 8. Program

```
package week3_module;
import java.util.Scanner;
public class MergeSortEasy {
    // Function to merge two sorted subarrays
    static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int[] leftArray = new int[n1];
        int[] rightArray = new int[n2];

        // Copy elements into temporary arrays
        for (int i = 0; i < n1; i++) {
            leftArray[i] = arr[left + i];
        }
        for (int j = 0; j < n2; j++) {
            rightArray[j] = arr[mid + 1 + j];
        }
        int i = 0;
        int j = 0;
        int k = left;
```

```

// Merge the temporary arrays
while (i < n1 && j < n2) {
    if (leftArray[i] <= rightArray[j]) {
        arr[k] = leftArray[i];
        i++;
    } else {
        arr[k] = rightArray[j];
        j++;
    }
    k++;
}
// Copy remaining elements of leftArray
while (i < n1) {
    arr[k] = leftArray[i];
    i++;
    k++;
}

// Copy remaining elements of rightArray
while (j < n2) {
    arr[k] = rightArray[j];
    j++;
    k++;
}
}

// Recursive Merge Sort function
static void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        // Sort left half
        mergeSort(arr, left, mid);

        // Sort right half
        mergeSort(arr, mid + 1, right);

        // Merge both halves
        merge(arr, left, mid, right);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter the number of placed students whose salary package values need to be
        sorted : ");

    int n = sc.nextInt();
    if(n < 1 || n > 100000) {
        System.out.println("Error: n must be between 1 and 100000.");
        sc.close();
        return;
    }
    int[] arr = new int[n];
    System.out.print("Enter the salary package values of " + n + " placed students : ");
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
        if (arr[i] < 100 || arr[i] > 10000) {
            System.out.println("Error: Score must be between 100 and 10000.");
            sc.close();
            return;
        }
    }
}

```

```

    }

    mergeSort(arr, 0, n - 1);
    System.out.print("Sorted salary package values of " + n + " placed students : ");
    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }
    sc.close();
}
}

```

#### 9. Program's output

Enter the number of placed students whose salary package values need to be sorted : 6

Enter the salary package values of 6 placed students : 450

1200

800

650

300

1500

Sorted salary package values of 6 placed students : 300 450 650 800 1200 1500

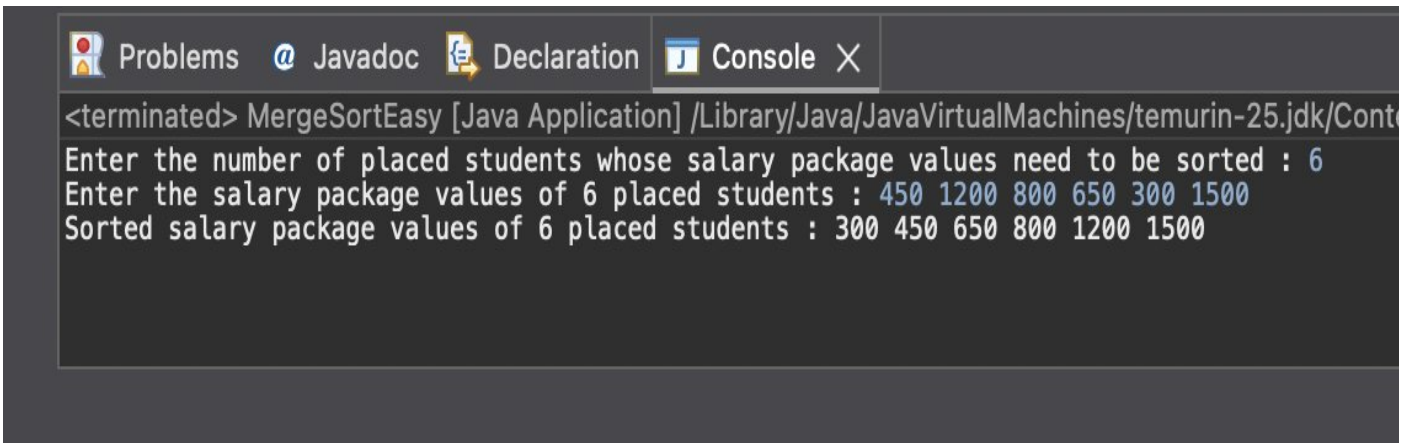
#### 10. Screenshot

program

```
1 package week3_module;
2 import java.util.Scanner;
3 public class MergeSortEasy {
4
5     // Function to merge two sorted subarrays
6     static void merge(int[] arr, int left, int mid, int right) {
7         int n1 = mid - left + 1;
8         int n2 = right - mid;
9         int[] leftArray = new int[n1];
10        int[] rightArray = new int[n2];
11
12        // Copy elements into temporary arrays
13        for (int i = 0; i < n1; i++) {
14            leftArray[i] = arr[left + i];
15        }
16        for (int j = 0; j < n2; j++) {
17            rightArray[j] = arr[mid + 1 + j];
18        }
19        int i = 0;
20        int j = 0;
21        int k = left;
22
23        // Merge the temporary arrays
24        while (i < n1 && j < n2) {
25            if (leftArray[i] <= rightArray[j]) {
26                arr[k] = leftArray[i];
27                i++;
28            } else {
29                arr[k] = rightArray[j];
30                j++;
31            }
32            k++;
33        }
34
35        // Copy remaining elements of leftArray
36        while (i < n1) {
37            arr[k] = leftArray[i];
38            i++;
39            k++;
40        }
41
42        // Copy remaining elements of rightArray
43        while (j < n2) {
44            arr[k] = rightArray[j];
45            j++;
46            k++;
47        }
48    }
```

```
49
50 // Recursive Merge Sort function
51 static void mergeSort(int[] arr, int left, int right) {
52
53     if (left < right) {
54
55         int mid = left + (right - left) / 2;
56
57         // Sort left half
58         mergeSort(arr, left, mid);
59
60         // Sort right half
61         mergeSort(arr, mid + 1, right);
62
63         // Merge both halves
64         merge(arr, left, mid, right);
65     }
66 }
67 public static void main(String[] args) {
68
69     Scanner sc = new Scanner(System.in);
70     System.out.print("Enter the number of placed students whose salary package values need to be sorted : ");
71     int n = sc.nextInt();
72     if(n < 1 || n > 100000)
73     {
74         System.out.println("Error: n must be between 1 and 100000.");
75         sc.close();
76         return;
77     }
78     int[] arr = new int[n];
79     System.out.print("Enter the salary package values of " + n + " placed students : ");
80     for (int i = 0; i < n; i++) {
81         arr[i] = sc.nextInt();
82         if (arr[i] < 100 || arr[i] > 10000) {
83             System.out.println("Error: Score must be between 100 and 10000.");
84             sc.close();
85             return;
86         }
87     }
88     mergeSort(arr, 0, n - 1);
89     System.out.print("Sorted salary package values of " + n + " placed students : ");
90     for (int i = 0; i < n; i++) {
```

program's output



```
<terminated> MergeSortEasy [Java Application] /Library/Java/JavaVirtualMachines/temurin-25.jdk/Cont
Enter the number of placed students whose salary package values need to be sorted : 6
Enter the salary package values of 6 placed students : 450 1200 800 650 300 1500
Sorted salary package values of 6 placed students : 300 450 650 800 1200 1500
```

## 11. EXPLANATION :-

### Program Breakdown

When the program starts, Java first looks for the main() method because it is the entry point of every Java program.

Execution starts here:

```
public static void main(String[] args)
```

### Step 1 : Create Scanner Object

```
Scanner sc = new Scanner(System.in);
```

### Purpose

Scanner is used to take input from the keyboard.

Without Scanner, the program cannot read the user's input.

### Step 2 : Read Number of Elements

```
int n = sc.nextInt();
```

Suppose input is

6

Then

Meaning there are six numbers to sort.

### Step 3 : Create Array

```
int[] arr = new int[n];
```

Since

n = 6

Java creates

```
arr = [ _, _, _, _, _, _ ]
```

Currently every position is empty.

### Step 4 : Read Array Elements

```
for(int i=0;i<n;i++)  
{  
    arr[i]=sc.nextInt();  
}
```

Iteration 1

```
arr[0]=450
```

Array

```
450 _ _ _ _ _
```

Iteration 2

```
arr[1]=1200
```

Array

```
450 1200 _ _ _ _
```

This continues until

```
450 1200 800 650 300 1500
```

```
if(arr[i] < 100 || arr[i] > 10000)
```

Purpose:

- Checks whether the entered package satisfies the given constraints.
- Displays an error message if an invalid package no is entered.

- Terminates the program to prevent invalid data from being processed.

#### **Step 5 : Call Merge Sort**

```
mergeSort(arr,0,n-1);
```

Since

$n=6$

This becomes

```
mergeSort(arr,0,5);
```

Meaning

Sort the array starting from index **0** to index **5**.

Control now moves from **main()** to

```
mergeSort()
```

#### **mergeSort() Method**

```
static void mergeSort(int[] arr,int left,int right)
```

This method performs the recursive division.

#### **Step 6 : Base Condition**

```
if(left<right)
```

If  $left==right$

Only one element exists.

Example

[450]

One element is already sorted.

So recursion stops.

#### **Step 7 : Find Middle Element**

```
int mid=left+(right-left)/2;
```

Initially

```
left=0
```

```
right=5
```

Calculation :  $\text{mid} = 0 + (5 - 0) / 2$

$\text{mid} = 2$

Now

Left Half :-  $0 \rightarrow 2$

Right Half :-  $3 \rightarrow 5$

Array

450 1200 800 | 650 300 1500

#### **Step 8 : Recursive Left Call**

`mergeSort(arr, left, mid);`

becomes

`mergeSort(arr, 0, 2);`

Again

0 1 2

gets divided.

Eventually

450

1200

800

Every element becomes individual.

#### **Step 9 : Recursive Right Call**

`mergeSort(arr, mid+1, right);`

becomes

`mergeSort(arr, 3, 5);`

Similarly

650

300

1500



become separate elements.

At this point Every subarray contains only one element. Now division stops.

### **merge() Method**

Now recursion starts returning. Instead of dividing, Java starts merging.

#### **Step 10 : Call merge()**

```
merge(arr,left,mid,right);
```

Merge 450 1200

#### **Step 11 : Find Sizes**

```
int n1=mid-left+1;
```

```
int n2=right-mid;
```

Suppose

```
left=0
```

```
mid=0
```

```
right=1
```

Then

```
n1=1
```

```
n2=1
```

Meaning :- Both temporary arrays contain one element.

#### **Step 12 : Create Temporary Arrays**

```
int[] leftArray=new int[n1];
```

```
int[] rightArray=new int[n2];
```

Example

```
leftArray :- 450
```

```
rightArray :- 1200
```

#### **Step 13 : Copy Elements**

```
for(...)
```

copies 450 into leftArray

Second loop copies

1200 into rightArray

Now original data is safely stored.

#### **Step 14 : Compare Elements**

Variables

i=0

j=0

k=left

Since

450<1200

Store

450 into arr[k]

Increment

i++

k++

Next

Left array finishes.

Remaining 1200 gets copied.

Now

450 1200 is sorted.

The same process repeats for

800 650 300 1500

Eventually larger sorted arrays are formed.

Finally :- 300 450 650 800 1200 1500

#### **Back to main()**

Once the entire array is sorted,

control returns to main()

#### **Step 15 : Print Array**

```
for(int i=0;i<n;i++)
```

```
prints
```

```
300 450 650 800 1200 1500
```

#### **Step 16 : Close Scanner**

```
sc.close();
```

The Scanner object is closed to free the input resource. The program then terminates.

#### **Overall Program Flow**

```
main()
```

↓

```
Read N
```

↓

```
Create Array
```

↓

```
Read Elements
```

↓

```
Call mergeSort()
```

↓

```
Divide Array Recursively
```

↓

```
Single Element Subarrays
```

↓

```
Call merge()
```

↓

```
Compare Elements
```

↓

```
Merge in Ascending Order
```

↓

```
Complete Array Sorted
```



Return to main()



Print Sorted Array



End Program